Higher-Rank Syntax as a Relative Monad

Code companion for `HigherRankSyntax/HigherRankSyntax/Equations.lean`

1 Purpose and influence

This note explains the mathematical content of `HigherRankSyntax/HigherRankSyntax/Equations.lean`. It is written in the same spirit as the development of syntactic transformations in Anja Petkovic Komel's thesis, especially the part beginning with the desiderata for syntactic transformations and continuing through the relative-monad proof for syntax.[1]

The guiding idea is the same. First one defines a purely syntactic action on expressions. Then one proves that the action is compatible with the other syntactic operations, especially instantiation/substitution. Finally the relative monad laws become short consequences of those interaction lemmas.

Our setting is a higher-rank generalization. The role of a signature or metacontext is played by a telescope-shaped object $\Gamma : \mathsf{Shape}C$. A slot $x : \Gamma \ni \alpha$ has an arity α, and each binder $j : C.\mathsf{Binder}\,\alpha$ carries its own arity $j.\mathsf{arity}$. Thus the application case is not simply a finite list of ordinary arguments: each argument lives below a fresh binder specified by the head arity. This is the source of most of the proof engineering in `Equations.lean`.

2 Relative monads for this syntax

The general relative-monad pattern is as follows. The functor J gives the generators of a syntactic context, while T gives expressions over that context. The unit embeds a generator as its generic expression. A Kleisli map assigns to each generator an expression in a target context, and its Kleisli extension is the action of that assignment on all expressions.

Definition 1 (Higher-rank generators and expressions)**.** *For a shape $\Gamma : \mathsf{Shape}C$, the generators of arity α are slots*

$$J\Gamma(\alpha) = \{x \mid x : \Gamma \ni \alpha\}.$$

The corresponding expressions are

$$T\Gamma(\alpha) = \mathsf{Expr}(\Gamma :: \alpha).$$

The unit sends a slot to its eta-expansion:

$$\eta_\Gamma(x) = \mathit{Expr.eta}\; x.$$

Definition 2 (Kleisli substitutions)**.** *A Kleisli map from Γ to Δ is a family*

$$f : \prod_\alpha (\Gamma \ni \alpha) \to \mathsf{Expr}(\Delta :: \alpha).$$

[1] `https://repozitorij.uni-lj.si/Dokument.php?id=152351&lang=slv`

In the Lean code this is wrapped as

$$\texttt{toSubst}\ f : \mathsf{Subst}\, C\, \mathsf{Shape.nil}\, \Gamma\, \Delta.$$

The more general record

$$\sigma : \mathsf{Subst}\, C\, \mathsf{pre}\, \mathsf{dom}\, \mathsf{cod}$$

keeps a fixed prefix pre*, replaces slots from* dom*, and leaves the prefix slots untouched:*

$$\sigma(x) : \mathsf{Expr}(\mathsf{pre} +\!\!+ \mathsf{cod} :: \alpha) \qquad (x : \mathsf{dom} \ni \alpha).$$

Definition 3 (Substitution action)**.** *For*

$$\sigma : \mathsf{Subst}\, C\, \mathsf{pre}\, \mathsf{dom}\, \mathsf{cod}$$

and a current binder depth τ*, the action is*

$$\sigma^*_\tau(-) : \mathsf{Expr}(\mathsf{pre} +\!\!+ \mathsf{dom} +\!\!+ \tau) \to \mathsf{Expr}(\mathsf{pre} +\!\!+ \mathsf{cod} +\!\!+ \tau).$$

In Lean this is `Subst.act`*. The depth* τ *records binders already entered by the recursive traversal.*

Mathematically, `Subst.act` has three cases for an application head in $\mathsf{pre} +\!\!+ \mathsf{dom} +\!\!+ \tau$.

- If the head comes from τ, it is locally bound and is preserved.
- If the head comes from dom, the substitution fires.
- If the head comes from pre, it is outside the substitution domain and is only weakened through the new codomain.

The Lean classifier `SubstSite` names exactly these cases:

`tau`, `dom`, `pre`.

The associated computation lemmas are `Subst.act_ap_inr`, `Subst.act_ap_inl_dom`, and `Subst.act_ap_inl_pre`.

3 Unit laws

The first two relative-monad equations are the two unit laws. In the thesis style, these are proved after isolating the fact that the generic application acts like the identity transformation.

Lemma 1 (Action preserves eta at a bound head)**.** *Let* $x : \tau \ni \alpha$*. Then*

$$\sigma^*_{\tau::\alpha}(\eta_{\mathsf{pre}+\!\!+\mathsf{dom}+\!\!+\tau}(x)) = \eta_{\mathsf{pre}+\!\!+\mathsf{cod}+\!\!+\tau}(x).$$

This is `Subst.act_eta_inr`*.*

Lemma 2 (Identity one-binder instantiation)**.** *The identity instantiation for the singleton telescope* $\langle\alpha\rangle$ *acts as the identity on expressions. This is the private recursion* `idOf`*, packaged as* `Subst.act_inst.id`*.*

Lemma 3 (Beta for eta)**.** *Instantiating the eta-expansion of the singleton head by a one-binder kit returns the corresponding kit application. This is* `Subst.act_inst.eta`*.*

Theorem 1 (Right unit)**.** *For every expression* $e : \mathsf{Expr}(\Gamma +\!\!+ \tau)$*,*

$$\mathit{Subst.id}\,\Gamma^*_\tau(e) = e.$$

This is `Subst.act_id`*.*

Theorem 2 (Left unit)**.** *For a Kleisli map* f *and slot* $p : \Gamma \ni \alpha$*,*

$$\mathit{toSubst}\, f^*_{\langle\alpha\rangle}(\eta(p)) = f(p).$$

This is `Subst.act_eta`*.*

4 The main interaction lemma

The thesis proves that a syntactic transformation commutes with instantiation before using that fact to prove composition. The direct analogue in this file is the one-binder diamond:

$$\begin{array}{ccc} \mathsf{Expr}((\mathsf{pre} +\!\!+ \mathsf{dom} +\!\!+ \tau :: \alpha) +\!\!+ \mathsf{ofList}(\upsilon)) & \xrightarrow{\sigma^*_{(\tau::\alpha)+\!\!+\mathsf{ofList}(\upsilon)}(-)} & \mathsf{Expr}((\mathsf{pre} +\!\!+ \mathsf{cod} +\!\!+ \tau :: \alpha) +\!\!+ \mathsf{ofLis} \\ \downarrow \kappa & & \downarrow \kappa' \\ \mathsf{Expr}(\mathsf{pre} +\!\!+ \mathsf{dom} +\!\!+ (\tau +\!\!+ \mathsf{ofList}(\rho)) +\!\!+ \mathsf{ofList}(\upsilon)) & \xrightarrow{\sigma^*_{(\tau+\!\!+\mathsf{ofList}(\rho))+\!\!+\mathsf{ofList}(\upsilon)}(-)} & \mathsf{Expr}(\mathsf{pre} +\!\!+ \mathsf{cod} +\!\!+ (\tau +\!\!+ \mathsf{ofList}(\rho)) +\!\!+ \end{array}$$

Here κ instantiates the singleton α-slot with fillers ι, while κ' instantiates it with the pointwise σ-acted fillers.

Theorem 3 (One-binder action/instantiation diamond)**.** *Acting by* σ *and then instantiating the singleton* α*-slot by the acted fillers agrees with first instantiating that singleton slot and then acting by* σ*. In Lean:*

$$\mathit{OneDiamond.actThenInst}\ \sigma\ \rho\ \upsilon\ \iota\ e = \mathit{OneDiamond.instThenAct}\ \sigma\ \rho\ \upsilon\ \iota\ e.$$

This is `oneDiamondAt`*.*

The proof has five head cases, and these are the conceptual heart of the file.

Case	Meaning	Proof behavior
`under`	the head is in $\mathsf{ofList}(\upsilon)$	preserve head, recurse on arguments
`alpha`	the head is the singleton being instantiated	descend into the selected filler
`tau`	the head is in the ambient depth τ	preserve head, recurse on arguments
`dom`	the head is substituted by σ	call the lifted companion
`pre`	the head is in the fixed prefix	preserve/weaken head, recurse on arguments

The `dom` case is the reason the proof is mutual. Substituting a dom-head exposes an expression $\sigma(z)$, and the instantiation around that expression must be rearranged. This is handled by the companion lemma.

Theorem 4 (Lifted one-binder diamond)**.** *Applying one singleton instantiation and then a second singleton instantiation agrees with one fused singleton instantiation whose fillers already include the first instantiation. In Lean:*

$$\mathit{OneLift.sequential}\ \rho\ \upsilon\ \chi\ \iota\ \eta\ e = \mathit{OneLift.fused}\ \rho\ \upsilon\ \chi\ \iota\ \eta\ e.$$

This is `oneLiftAt`*.*

The companion has three head cases:

$$\texttt{under}, \qquad \texttt{beta}, \qquad \texttt{pre}.$$

The `beta` case calls back into `oneDiamondAt`; ordinary argument cases recurse structurally.

5 Composition

The substitution-level composition operation is

$$\texttt{Subst.comp}\ \sigma\ \theta : \mathsf{Subst}\, C\, \mathsf{pre}\, \mathsf{dom}\, \mathsf{cod},$$

where

$$(\texttt{Subst.comp}\ \sigma\ \theta)(x) = \theta^*_{\langle\alpha\rangle}(\sigma(x)) \qquad (x : \mathsf{dom} \ni \alpha).$$

This is the general form of Kleisli composition, before specializing to empty prefixes.

Theorem 5 (Action preserves substitution composition)**.** *For arbitrary substitutions*

$$\sigma : \mathsf{Subst}\, C\, \mathsf{pre}\, \mathsf{dom}\, \mathsf{mid}, \qquad \theta : \mathsf{Subst}\, C\, \mathsf{pre}\, \mathsf{mid}\, \mathsf{cod},$$

and every expression $e : \mathsf{Expr}(\mathsf{pre} +\!\!+ \mathsf{dom} +\!\!+ \tau)$*,*

$$\textit{Subst.comp}\ \sigma\ \theta^*_\tau(e) = \theta^*_\tau(\sigma^*_\tau(e)).$$

This is `Subst.act_comp`*.*

The proof follows the three head cases for `Subst.act`. The `tau` and `pre` cases are preserved-head cases. The `dom` case is the only substantive one: after σ substitutes the head, one must commute θ past the one-binder instantiation generated by the arguments. In the current file that bridge is still routed through `interchange`; conceptually it should be a direct use of `oneDiamondAt`.

Corollary 1 (Kleisli composition law)**.** *For Kleisli maps* $f : \Gamma \to T\Delta$ *and* $g : \Delta \to T\Xi$*,*

$$\textit{toSubst}(\textit{Subst.kcomp}\ f\ g)^*_\tau(e) = \textit{toSubst}\ g^*_\tau(\textit{toSubst}\ f^*_\tau(e)).$$

This is `Subst.act_kcomp`*, now a short corollary of* `Subst.act_comp`*.*

6 Code map

Lean declaration	Mathematical role	Thesis-style analogue
`SubstSite`	Classifies an action head as τ, dom, or pre.	Case split for syntax-directed action.
`Subst.act_ap_inr`	Computation when the head is bound by the current depth.	Variable/binder preservation case.
`Subst.act_ap_inl_dom`	Computation when substitution fires at a domain head.	Nontrivial substitution case.

`Subst.act_ap_inl_pre`	Computation when the head is outside the domain.	Parameter/prefix preservation case.
`Subst.act_inst.eta`	Beta law for eta-expanded singleton heads.	Generic application computes correctly.
`Subst.act_inst.id`	Identity singleton instantiation acts as the identity.	Identity instantiation is trivial.
`Subst.act_id`	Identity substitution acts as identity.	Relative monad law $\eta^\dagger = \mathsf{id}$.
`Subst.act_eta`	Action of a Kleisli map on an eta-generator returns the chosen filler.	Relative monad law $f^\dagger \circ \eta = f$.
`oneDiamondAt`	Action commutes with one-binder instantiation.	Action interacts with instantiation.
`oneLiftAt`	Lifted companion needed by the domain branch of `oneDiamondAt`.	Compatibility of nested instantiations.
`preNaturalityAt`	Prefix-specialized naturality theorem still present as a separate recursive proof.	A derived specialization, not conceptually primitive.
`interchange`	Current bridge from the one-binder diamond to the composition proof.	Should become a wrapper around the interaction lemma.
`Subst.comp`	Substitution-level Kleisli composition.	Composition of syntactic transformations.
`Subst.act_comp`	Action preserves arbitrary substitution composition.	Action preserves composition.
`Subst.act_kcomp`	Kleisli composition law for `toSubst`.	Third relative-monad law.
`InterchangeFuel`	Well-founded measure for mutual one-binder interaction.	Proof-technical structural induction measure.

7 What the Lean file should eventually say

The current file already proves the right mathematical facts, but its remaining organization still reflects the path by which the proof was found. The clean story should be:

1. define the action and prove its three head computations;
2. prove the eta and identity-instantiation facts;
3. prove the one-binder interaction lemma, with its lifted companion;
4. prove arbitrary substitution composition;
5. specialize to the relative monad laws.

In that presentation, `PreNaturality` and `Interchange` are not foundational layers. They are special cases of the action/instantiation interaction theorem, kept only until the one-binder theorem is generalized enough for `Subst.act_comp` to call it directly.